

Trabajo Práctico Integrador — Sistema de Gestión de Turnos Médicos

Programación 2 — Proyecto integrador final de la carrera

Contexto

Un consultorio médico que hoy gestiona su agenda de forma manual (las secretarías cargan los turnos y los imprimen antes de la jornada) decide digitalizar su operación mediante una aplicación web. La clínica está creciendo y necesita una solución que acompañe ese crecimiento.

El sistema deberá dar soporte a la operación diaria de la clínica: la gestión de sus usuarios, de la agenda médica y de los turnos de los pacientes. El alcance concreto de cada parte lo van a ir conociendo a medida que avance el proyecto.

Organización del trabajo

Este es el segundo trabajo práctico de la materia: se espera que apliquen de forma integrada todo lo aprendido hasta el momento más los conceptos de Angular que se irán desarrollando a lo largo del cuatrimestre.

El desarrollo se divide en dos etapas:

Backend (4 semanas) — Node.js + Express, sobre la base de datos que les vamos a proveer.

Frontend (4 semanas) — Angular 21 + Angular Material, consumiendo la API que ustedes mismos construyeron.

Las consignas se liberan **de forma incremental**: cada semana van a recibir un nuevo documento con la parte del enunciado correspondiente a esa semana, y la entrega es igualmente semanal (con un máximo de 15 días entre una entrega y la siguiente). La idea es que se enfoquen en lo que toca en cada etapa, y que cada entrega se construya sobre la anterior sin adelantarse.

Stack tecnológico obligatorio

Backend: Node.js + Express, autenticación con JWT, hash de contraseñas con bcrypt.

Base de datos: MySQL/MariaDB — se les provee el script de base inicial al comenzar la etapa de backend.

Frontend: Angular 21 + Angular Material.

Documentación de API: Postman.

Conviene que desde el principio tengan el entorno de trabajo preparado (Node, un cliente de base de datos, el CLI de Angular) para no perder tiempo en la primera entrega.

Formato de respuesta del backend

Todos los servicios del backend deben responder con una **estructura uniforme**, tanto si la operación resultó exitosa como si falló. Como mínimo, cada respuesta debe incluir:

Un código numérico que identifique el resultado de la operación (por ejemplo, el código de estado HTTP correspondiente).

Un texto de estado que indique si la operación fue satisfactoria (tipo `"ok"`) o si falló (con un mensaje descriptivo del error).

Una propiedad de datos donde viajen los datos solicitados cuando el servicio los devuelva (por ejemplo, el resultado de una consulta). Cuando la operación no retorne datos, esta propiedad puede ir vacía o nula.

Ejemplo orientativo:

```
{  
  "codigo": 200,  
  "estado": "ok",  
  "datos": { ... }  
}
```

El nombre exacto de cada propiedad queda a criterio del grupo, siempre que se mantengan estos tres elementos y se respeten de forma **consistente en todos los endpoints**, desde la primera entrega.

Modalidad de entrega y evaluación

Cada semana van a recibir la consigna correspondiente y deberán entregar lo pedido para esa etapa, funcional y completo.

El trabajo se presenta el día establecido para la defensa; son responsables de contar con el material necesario (repositorio actualizado, base de datos, credenciales de prueba, etc.).

La entrega se realiza a través del aula virtual, con el link al repositorio utilizado.

El diseño visual y los estilos quedan a criterio de cada grupo, siempre respetando la estructura y las funciones solicitadas en cada consigna semanal.

Las contraseñas deben almacenarse siempre hasheadas (bcrypt); no se acepta texto plano en ningún momento del desarrollo.

Qué van a recibir de acá en más

Un documento semanal con la consigna de esa etapa (primero 4 de backend, luego 4 de frontend).

El script de base de datos inicial, al arrancar la etapa de backend.

Trabajen prolijo, versionen desde el primer día y consulten cuando lo necesiten.